

Acrambly: Perception

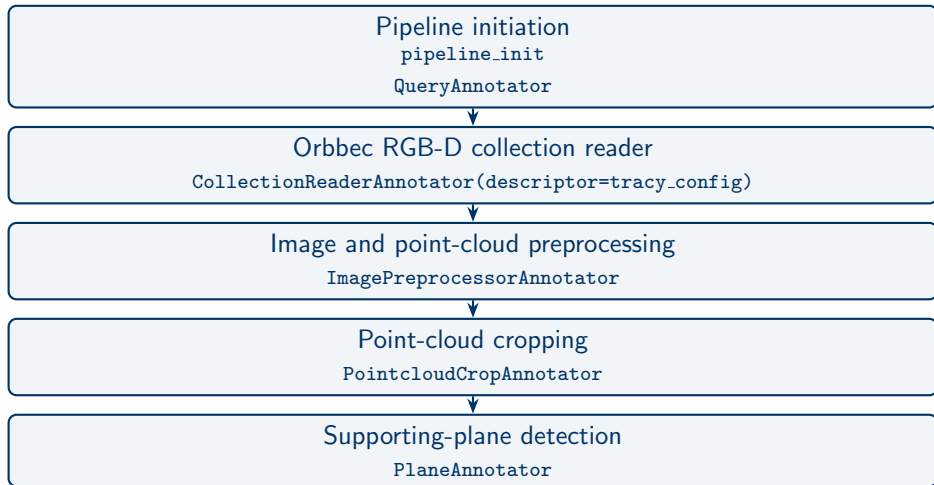
Mid-term Presentation

M. Nasim Palakka Valappil

Oct 2 2026

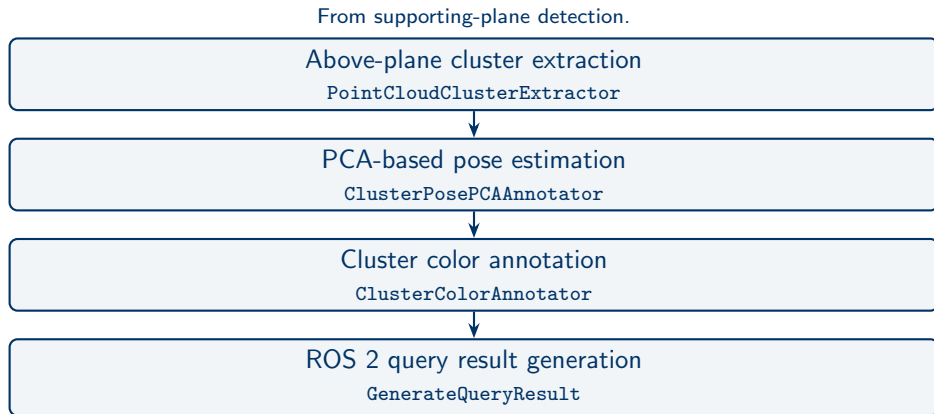
- 1 Perception pipeline 1: White
- 2 Perception pipeline 2: Blue
- 3 Storage Red
- 4 CRAM and RoboKudo integration

Perception Pipeline 1: White (1/2)



Continues with above-plane cluster extraction.

Perception Pipeline 1: White (2/2)

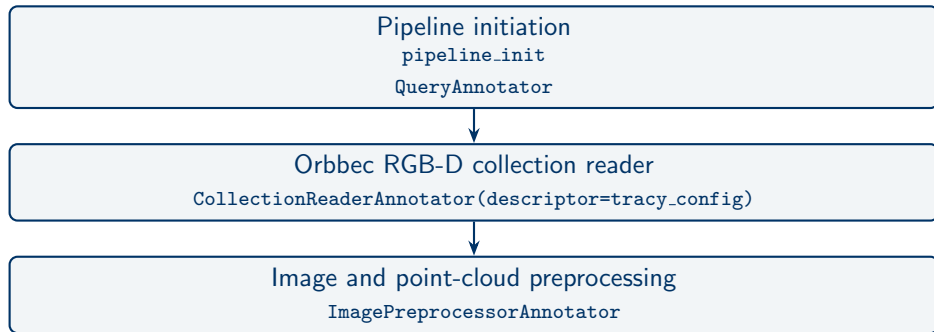


Observed behavior

The pipeline demonstrated the complete path from RGB-D acquisition to object hypotheses with pose and color information.

The pipeline was falsely detecting large box of grey and white (for reflective areas) of the table. While missing requested colored cubes.

Preception Pipeline 2: Blue(1/2)



Continues with query-selected HSV segmentation.

Preception Pipeline 2: Blue(2/2)

From image and point-cloud preprocessing.

Detect one colored cube at a time based on HSV range

`ImageClusterExtractor`



Fits an oriented bounding box

`ClusterPoseBBAnnotator`



ROS 2 query result generation

`GenerateQueryResult`

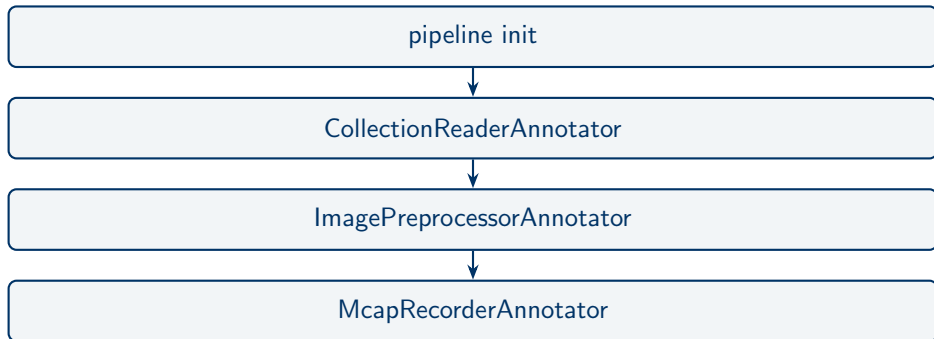
This pipeline reliably detected the queried colored cube and returned its bounding-box pose.
It precisely detect even the pose of smaller cubes.
However, it is a coustom solution for the specific task.

- **Motivation:** On the newer Linux kernel used for this project, the original RoboKudo MongoDB recording workflow encountered a compatibility problem.
- **Response:** A new `storage_red` analysis engine with a `McapRecorderAnnotator` custom annotator for recording sensor data without MongoDB.

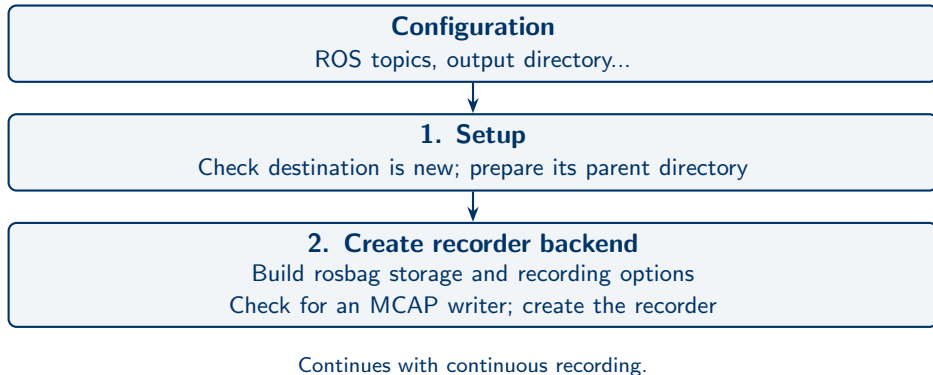
Storage Red: recorded topics

Topic source	Recorded information
Orbbec depth topic	Depth image
Orbbec color topic	RGB image
Orbbec camera-info topic	Camera calibration
/tf_static	Static transforms

Storage Red: processing flow



MCAP recorder: lifecycle (1/2)



MCAP recorder: lifecycle (2/2)

From recorder backend creation.

3. Continuous recording



4. Status updates



5. Shutdown and finalization

Storage Red: previous and current recording

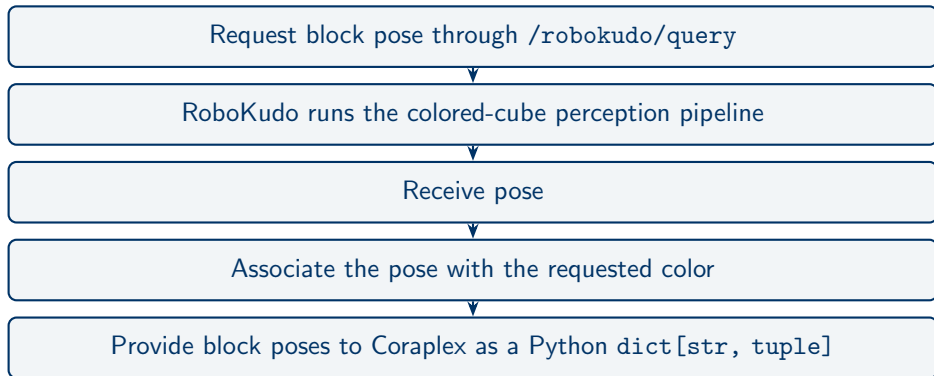
Aspect	Previous: <code>storage.py</code>	New: <code>storage_red.py</code>
Storage backend	MongoDB through <code>StorageWriter</code>	ROS 2 <code>rosbag2</code> with MCAP through <code>McapRecorderAnnotator</code>
Data saved	Processed CAS data, views, and world model state	Original RGB-D, camera-info, and transform ROS messages
Recording behavior	Writes the current CAS when the pipeline ticks	Records subscribed ROS topics continuously while the pipeline runs
Replay input	MongoDB records read back into a CAS	MCAP bag replayed as sensor topics
Advantage in the notes	More compressed	Easier implementation

MCAP recorder: observed behavior

- The analysis engine recorded the configured sensor stream to an MCAP bag.
- The data was replayed successfully.
- Many later experimentations was done using the data stored in next cloud with storage red.

Laptop demonstration: replay the MCAP bag.

CRAM and RoboKudo integration V1



Block-stacking integration without SemDtSync.

CRAM and RoboKudo integration V1

- **White pipeline:** Estimates all block poses at the same time in one perception pipeline run.
- **Blue pipeline:** Each ROS action goal requests object type `block` and one requested color. Found color blocks are retained; missing colors are queried again.
- **Action-future handling:** The query works with a standalone ROS node and with a node already managed by an executor.

Thank You

Questions?